

- 1) Derive a mathematical model of the tire, with the road surface position z_r as the input signal and the tire's position z_w as the output signal:

② Equations

1. Newton second law:
 $F_i = m_w \ddot{z}_w$

2. Forces
 Spring force: $F_c = c_w(z_w - z_r)$ $\begin{cases} c_w = \text{spring constant} \\ d_w = \text{damping constant} \end{cases}$
 Damping force: $F_d = d_w(\dot{z}_w - \dot{z}_r)$

3. Total forces
 $\sum F = F_i + F_c + F_d = 0 \Rightarrow m_w \ddot{z}_w + c_w(z_w - z_r) + d_w(\dot{z}_w - \dot{z}_r) = 0$

4. Differential equation:
 $m_w \ddot{z}_w + d_w \dot{z}_w + c_w z_w = c_w z_r + d_w \dot{z}_r$

5. Laplace
 $m_w s^2 z_w(s) + d_w s z_w(s) + c_w z_w(s) = c_w z_r(s) + d_w s z_r(s)$
 $G(s) = \frac{z_w(s)}{z_r(s)} = \frac{c_w + d_w s}{m_w s^2 + d_w s + c_w}$

6. Answer
 $G(s) = \frac{c_w + d_w s}{m_w s^2 + d_w s + c_w}$

* Sorry for the bad resolution, our scanning programs didn't want to work properly for us. If you can't read it, we can send you a new one or show you on a consultation lecture.

Matlab:

```
transfer_func = '(cw+dw*s)/((mw*s^2)+(dw*s)+cw)';
```

- 2) Use step response identification to identify the spring and damper coefficients for your tire using the step response signals:

Equations

② $M_p = \text{get it from matlab code} \Rightarrow$
 $\Rightarrow M_p = 0,5747$

$$M_p = e^{\frac{-\pi \xi}{\sqrt{1-\xi^2}}} \Rightarrow \ln(M_p) = -\frac{\pi \xi}{\sqrt{1-\xi^2}} \Rightarrow$$

$$-\ln(M_p) = \frac{\pi \xi}{\sqrt{1-\xi^2}} \Rightarrow \pi \xi = -\ln(M_p) \cdot \sqrt{1-\xi^2} \Rightarrow$$

$$\Rightarrow \xi = \frac{-\ln(M_p) \cdot \sqrt{1-\xi^2}}{\pi} \Rightarrow \xi^2 = \left(\frac{-\ln(M_p)}{\pi} \right)^2 \cdot (1-\xi^2) \Rightarrow$$

$$\Rightarrow \xi^2 + \left(\frac{-\ln(M_p)}{\pi} \right)^2 \cdot \xi^2 = \left(\frac{-\ln(M_p)}{\pi} \right)^2 \Rightarrow$$

$$\Rightarrow \xi^2 = \frac{\left(\frac{-\ln(M_p)}{\pi} \right)^2}{1 + \left(\frac{-\ln(M_p)}{\pi} \right)^2} \Rightarrow \xi = \frac{-\ln(M_p)}{\pi \sqrt{1 + \left(\frac{-\ln(M_p)}{\pi} \right)^2}} \Rightarrow$$

$$\Rightarrow M_p = 0,5764 \Rightarrow \xi = 0,1727$$

(matlab calculations)

$\Delta T = 0,0450$ (from matlab code)

$$\omega_n = \frac{\pi}{\Delta T \sqrt{1-\xi^2}} \Rightarrow \omega_n = 68,7636$$

(matlab calculations)

$$\omega_n = \sqrt{\frac{c_w}{m_w}} \Rightarrow c_w = \omega_n^2 \cdot m_w \Rightarrow c_w = 2,2696 \cdot 10^5$$

$$\xi = \frac{d_w}{2 \cdot \sqrt{c_w \cdot m_w}} \Rightarrow d_w = \xi \cdot 2 \sqrt{c_w \cdot m_w} \Rightarrow d_w = 1,1404 \cdot 10^3$$

matlab

Matlab:

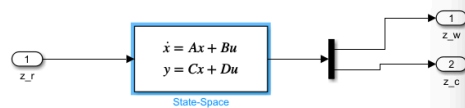
`cw=2.2696e+05;`

`dw=1.1404e+03;`

3) Implement the state-space model:

The problem is mostly solved in matlab but underneath you can see the simulink graphs and the state-space model.

State space-model from simulink:



State Space

State-space model:
 $\dot{x}/dt = Ax + Bu$
 $y = Cx + Du$

'Parameter tunability' controls the runtime tunability level for A, B, C, D.
'Auto': Allow Simulink to choose the most appropriate tunability level.
'Optimized': Tunability is optimized for performance.
'Unconstrained': Tunability is unconstrained across the simulation targets

Selecting the 'Allow non-zero values for D matrix initially specified as zero' checkbox requires the block to have direct feedthrough and may cause algebraic loops.

Parameters

A:
 $[0 \ 1 \ 0 \ 0; -(cw+cs)/mw \ -(dw+ds)/mw \ (cs/mw) \ (ds/mw); 0 \ 0 \ 0 \ 1; (c; \dots]$

B:
 $[0; cw/mw; 0; 0]$ $[0; 4728.4; 0; 0]$

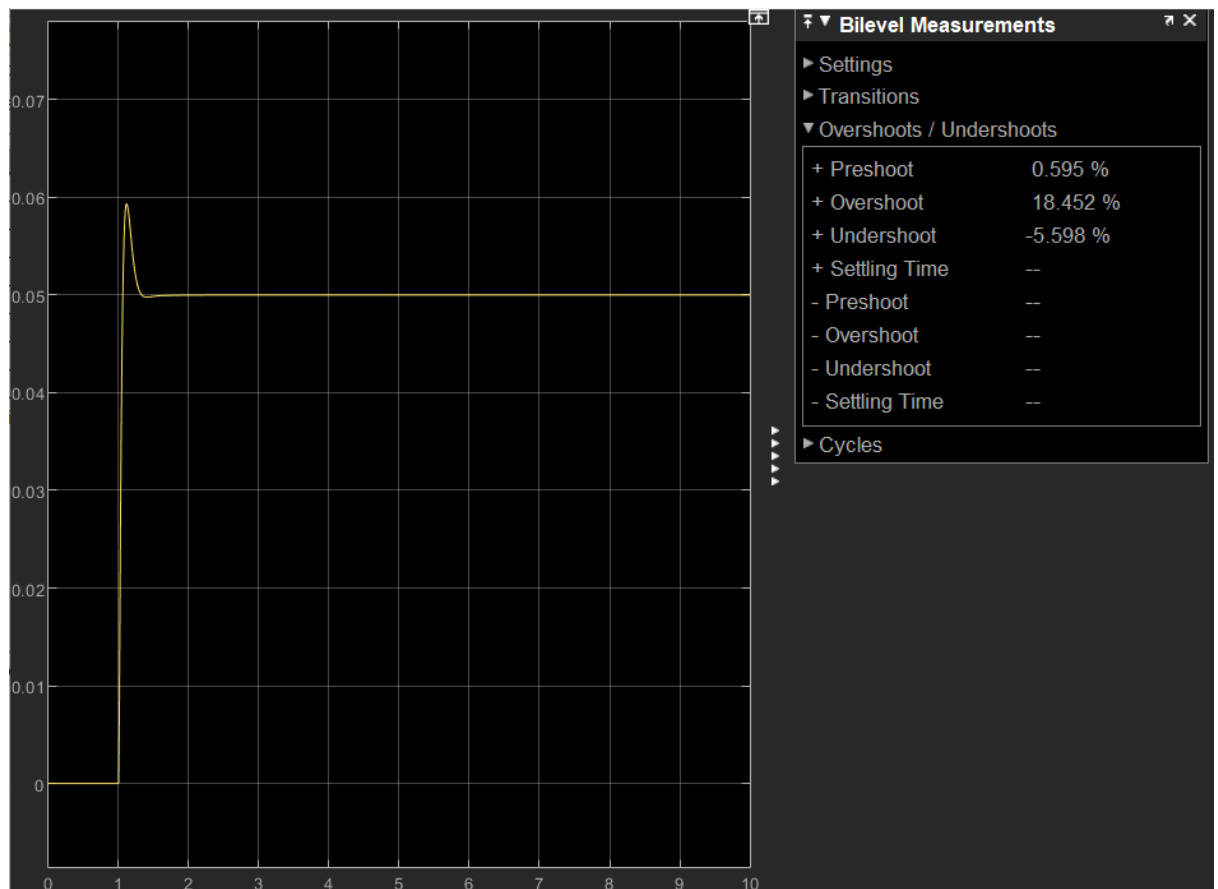
C:
 $[1 \ 0 \ 0 \ 0; 0 \ 0 \ 1 \ 0]$ $<2 \times 4 \text{ double}>$

D:
 $[0; 0]$ $[0; 0]$

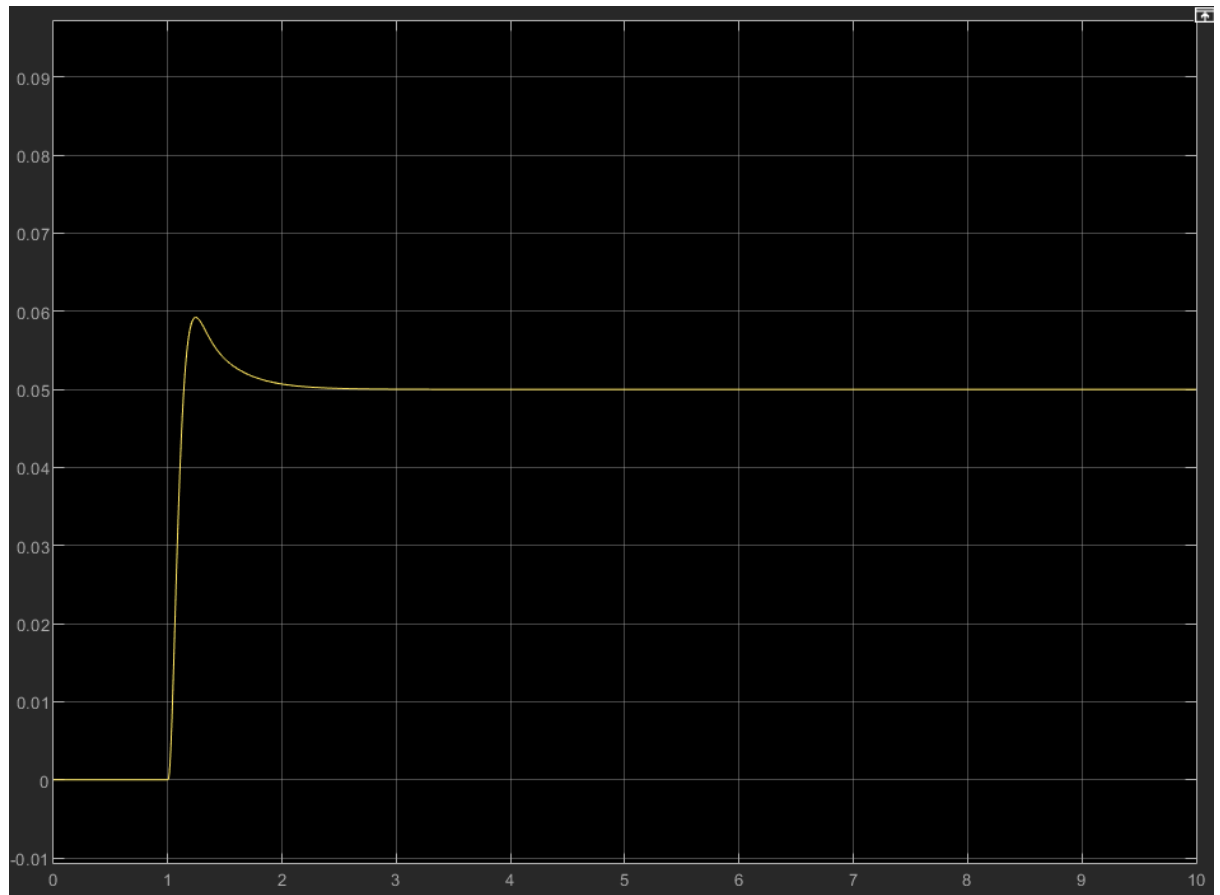
Initial conditions:

OK Cancel Help Apply

Graph of the wheel position from simulink:



Graph of the chassis position from simulink:



Matlab:

```
max_value = 0.0592;  
settling_time = 1.6253;
```

4) Determine the eigenvalues of the system, is the system stable?

This problem is done in matlab and the answers from matlab is shown below:

Matlab:

```
(1.0e+02 *)  
eig_a = -1.4157 + 0.0000i;  
eig_b = -0.1592 + 0.1045i;  
eig_c = -0.1592 - 0.1045i;  
eig_d = -0.0347 + 0.0000i;
```

System is stable (all the eigenvalues have negative real parts).

5) What conclusion can you draw from the placement of the poles of the system?

We talked with Jonas and he said that we only needed to answer with the alternative that we think is right. If that wouldn't be enough tell us and we will explain why we got the answer we got.

The correct answer is the first one "If the imaginary part is larger than the real part, the more oscillatory the system response becomes.":

- ☒ *If the imaginary part is larger than the real part, the more oscillatory the system response becomes.*
- ☐ *If the real part is larger than the imaginary part, the more oscillatory the system response becomes.*
- ☐ *The location of the poles does not matter as long as the poles are stable.*

6) Answer with the transformation matrix:

⑥ $Z_w = \text{wheel position}$ $\dot{Z}_w = \text{wheel velocity}$
 $Z_w - Z_c = \text{relative distance b/w wheel \& chassis}$
 $\dot{Z}_w - \dot{Z}_c = \text{relative velocity b/w wheel \& chassis}$

$Z = Tx$

$x = \text{old choice of state variables}$

from previous problem :- $x_1 = Z_w$ (wheel position)
 $x_2 = \dot{Z}_w$ (wheel velocity)
 $x_3 = Z_c$ (chassis position)
 $x_4 = \dot{Z}_c$ (chassis velocity).

$Z = \text{is new choice of state variables}$

$Z_1 = Z_w$ (wheel position)
 $Z_2 = \dot{Z}_w$ (wheel velocity)
 $Z_3 = Z_w - Z_c$ (relative distance b/w wheel \& chassis)
 $Z_4 = \dot{Z}_w - \dot{Z}_c$ (relative velocity b/w wheel \& chassis)

relationship b/w new \& old state variables

$$\begin{array}{lcl} Z_1 = x_1 & & Z_w = x_1 \\ Z_2 = x_2 & \Rightarrow & \dot{Z}_w = x_2 \\ Z_3 = x_1 - x_3 & & Z_w - Z_c = x_3 \\ Z_4 = x_2 - x_4 & & \dot{Z}_w - \dot{Z}_c = x_4 \end{array}$$

The transformation matrix $Z = Tx$

$$Z = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \Rightarrow T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}$$

Matlab:

$T = [1 \ 0 \ 0 \ 0; \ 0 \ 1 \ 0 \ 0; \ 1 \ 0 \ -1 \ 0; \ 0 \ 1 \ 0 \ -1]$

- 7) Augment the model output with the chassis acceleration and the relative distance between road and wheel, i.e., the system should now have 4 output signals:

Problem 7

$$\dot{x} = Ax + Bu$$

$$y = Cx + Du.$$

$\ddot{z}_c$ = chassis acceleration
 $z_w - z_r$ = relative distance b/w wheel & road.

from previous problems.

$$x_1 = z_w \text{ (wheel position)}$$

$$x_2 = \dot{z}_w \text{ (wheel velocity)}$$

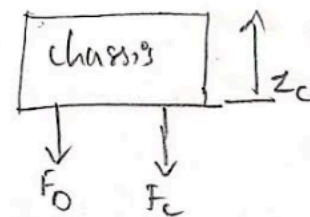
$$x_3 = z_c \text{ (chassis position)}$$

$$x_4 = \dot{z}_c \text{ (chassis velocity)}$$

The output equation is

$$y = C_{\text{new}}x + D_{\text{new}}u.$$

The free body diagram of chassis



$$m_c a_c = -F_D - F_c.$$

$$F_D = d_s (\dot{z}_c - \dot{z}_w)$$

$$F_c = c_s (z_c - z_w)$$

$$\Rightarrow m_c a_c = -F_D - F_c$$

$$a_c = \ddot{z}_c = \frac{1}{m_c} (-F_D - F_c).$$

$$\ddot{z}_c = \frac{1}{m_c} \times (-d_s (\dot{z}_c - \dot{z}_w) + (-c_s (z_c - z_w)))$$

$$= \frac{1}{m_c} \times [d_s (\dot{z}_w - \dot{z}_c) + c_s (z_w - z_c)]$$

$$\ddot{z}_c = \frac{1}{m_c} \times d_s (x_2 - x_4) + c_s (x_1 - x_3)$$

The relative distance between wheel and road,

$$z_w - z_r \Rightarrow x_1 - z_r$$

$$y_1 = z_w \text{ (wheel position)}$$

$$y_2 = z_c \text{ (chassis position)}$$

$$y_3 = \ddot{z}_c \text{ (chassis acceleration)}$$

$$y_4 = z_w - z_r \text{ (relative distance b/w wheel & road)}$$

The output equation,

$$y = C_{new} x + D_{new}$$

$$C_{new} x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ \frac{c_s}{m_c} & \frac{d_s}{m_c} & -\frac{c_s}{m_c} & -\frac{d_s}{m_c} \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

The Dnew matrix

output includes the effect of the road position z_r .
 $z_w - z_r$ (relative distance b/w wheel & road).

$$D_{new} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ -1 \end{bmatrix}$$

$$\Rightarrow C_{new} \text{ Matrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ \frac{c_s}{m_c} & \frac{d_s}{m_c} & -\frac{c_s}{m_c} & -\frac{d_s}{m_c} \\ 1 & 0 & 0 & 0 \end{bmatrix} \quad D_{new} \text{ matrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ -1 \end{bmatrix}$$

Matlab:

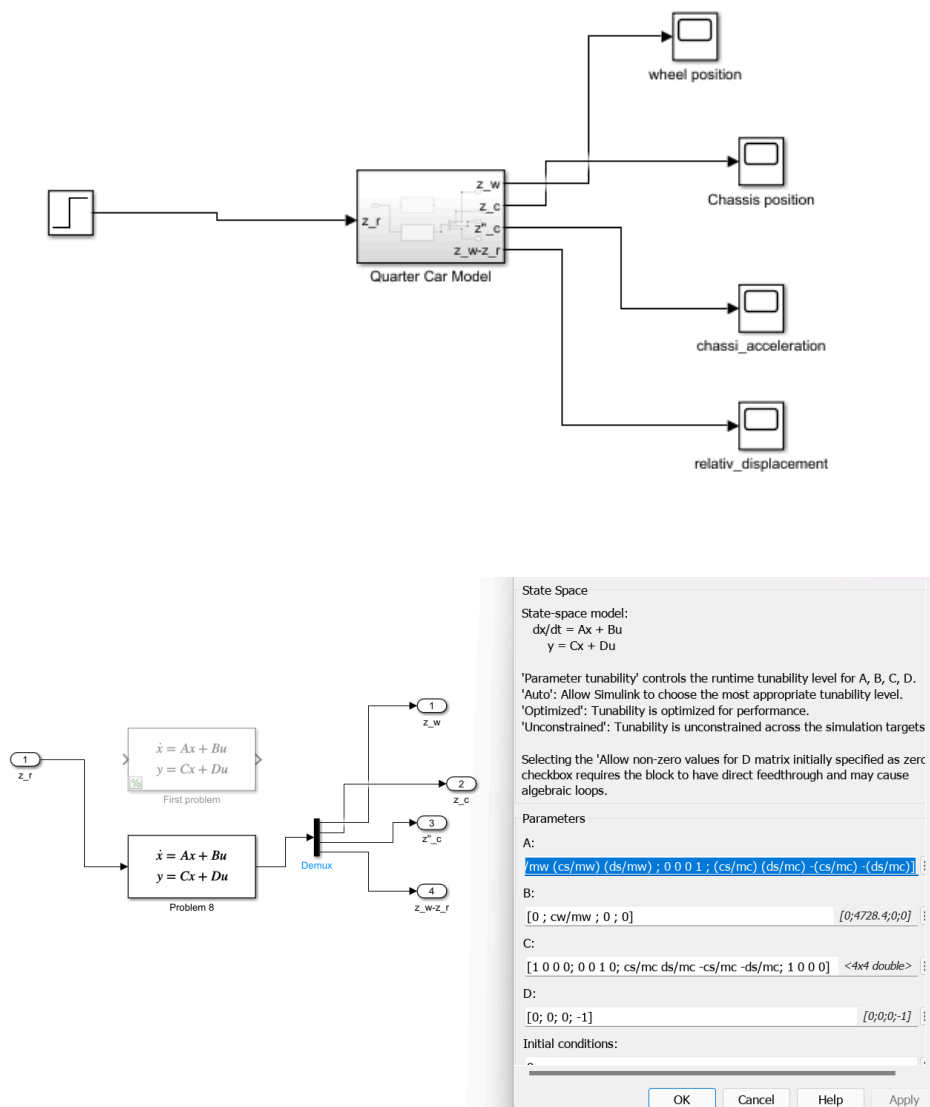
C = [1 0 0 0; 0 0 1 0; cs/mc ds/mc -cs/mc -ds/mc; 1 0 0 0]

D = [0; 0; 0; -1]

8) Find the parameter values, ds and cs , which maximizes road handling and ride comfort performance when traveling over the 5 cm speed bump:

This problem is solved in MATLAB (see the solutions below). We found that the lowest spring constant in the given range, 10,000 N/m, is the best, which suggests we should adjust our range from 10,000 - 60,000 N/m to something narrower. A smaller starting point in the range would give us more precise and accurate results. The damper constant falls on the lower end of the range but is still within acceptable limits, so the current range works well for it, but we can offcourse make the steps smaller than 500 on the damper to be more precise.

We changed our simulink for this problem, se below:



Solutions:

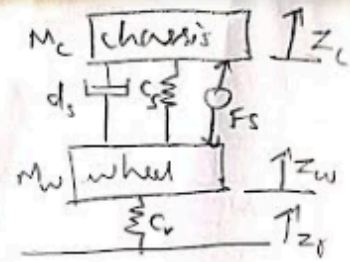
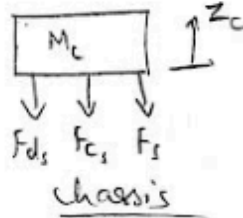
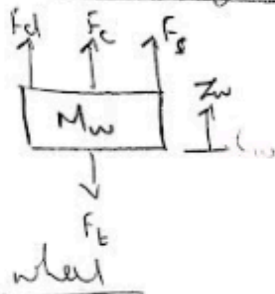
The optimal damping coefficient (ds) is: 2500.00 Ns/m

The optimal spring coefficient (cs) is: 10000.00 N/m

9) Determine the A, B, and H matrices:

Problem 9

Free body diagrams



F_s is the actuator force acting b/w wheel & chassis
 $F_s = \frac{1}{\tau} F_s + \frac{1}{\tau} u$ (τ - time constant = 0.1 s)

$$m_w a_w = F_d + F_c - F_t + F_s$$

$$m_c a_c = -F_{ds} - F_{cs} - F_s$$

$$a_w = \ddot{z}_w = \frac{1}{m_w} [-d_s(\dot{z}_c - \dot{z}_w) + (-c_s(z_c - z_w)) - c_w(z_w - z_g) + F_s]$$

$$a_c = \ddot{z}_c = \frac{1}{m_c} (-d_s(\dot{z}_c - \dot{z}_w)) + (-c_s(z_c - z_w)) - F_s$$

State vector $x_c = [x \ F_s]^T \Rightarrow \begin{bmatrix} x \\ F_s \end{bmatrix}$

$$x_c = \begin{bmatrix} z_w \\ \dot{z}_w \\ z_c \\ \dot{z}_c \\ F_s \end{bmatrix}$$

($x = x_1, x_2, x_3, x_4$ from the previous problem)

$$\dot{x}_e = A x_e + B u + H d$$

$$x_e = A \cdot \begin{bmatrix} x \\ F_s \end{bmatrix}$$

$$x_e = A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ \frac{-c_w + c_s}{m_w} & \frac{-d_w + d_s}{m_w} & \frac{c_s}{m_w} & \frac{d_s}{m_w} & \frac{1}{m_w} & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ \frac{c_s}{m_c} & \frac{d_s}{m_c} & \frac{-c_s}{m_c} & \frac{-d_s}{m_c} & \frac{-1}{m_c} & \frac{-1}{\tau} \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} z_w \\ \dot{z}_w \\ z_c \\ \dot{z}_c \\ F_s \end{bmatrix} +$$

$$B = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \frac{1}{\tau} \end{bmatrix} u$$

$$H = \begin{bmatrix} 0 \\ \frac{c_w}{m_w} \\ 0 \\ 0 \\ 0 \end{bmatrix} d.$$

*In the B and H matrix please do neglect the u and d and only consider the matrices (we only use them in the equations).

Matlab:

```
A_active = '[0 1 0 0 0; -(cw+cs)/mw -(dw+ds)/mw cs/mw ds/mw 1/mw; 0 0 0 1 0; cs/mc ds/mc -cs/mc -ds/mc -1/mc; 0 0 0 0 -1/zeta]';
B_active = '[0; 0; 0; 0; 1/zeta]';
H_active = '[0; cw/mw; 0; 0; 0]';
```